

强化学习第五次作业

2312167-刘振毫

摘要

本实验实现了两种时序差分 (Temporal Difference, TD) 学习方法求解 CartPole 马尔可夫决策过程问题: SARSA 算法和 Q-learning 算法。实验基于 OpenAI Gym 的 CartPole-v1 环境, 通过离散化状态空间构建 MDP 模型, 分别实现了两种 TD 控制算法, 并对它们的性能进行了全面比较。实验结果表明, Q-learning 算法在真实环境评估中表现更优, 平均回合奖励达到 445.33, 而 SARSA 算法的平均奖励为 247.24。Q-learning 作为 off-policy 方法, 能够直接学习最优策略, 收敛到更高的状态值函数; SARSA 作为 on-policy 方法, 考虑实际执行的动作, 策略更加保守。本报告详细分析了两种方法的理论基础、实现细节和性能差异, 为强化学习中 TD 方法的应用提供了重要参考。

关键词: 时序差分学习; SARSA; Q-learning; 小车倒立摆; on-policy; off-policy

Abstract

This experiment implements two Temporal Difference (TD) learning methods to solve the CartPole Markov Decision Process problem: SARSA algorithm and Q-learning algorithm. Based on the OpenAI Gym CartPole-v1 environment, the experiment constructs an MDP model by discretizing the state space, implements two TD control algorithms, and comprehensively compares their performance. Experimental results show that the Q-learning algorithm performs better in real environment evaluation, achieving an average episode reward of 445.33, while the SARSA algorithm achieves an average reward of 247.24. Q-learning, as an off-policy method, can directly learn the optimal policy and converge to a higher state value function; SARSA, as an on-policy method, considers the actually executed actions and has a more conservative policy. This report provides detailed analysis of the theoretical foundations, implementation details, and performance differences of the two methods, offering important references for the application of TD methods in reinforcement learning.

Key Words: Temporal Difference Learning; SARSA; Q-learning; CartPole; on-policy; off-policy

1 实验目的

本实验旨在通过实现和比较两种时序差分学习方法，深入理解强化学习中无模型控制算法的核心原理和应用场景。具体目标包括：

1. 掌握时序差分学习的基本原理，理解其结合蒙特卡洛和动态规划的优势
2. 理解 SARSA 算法的 on-policy 特性和更新机制
3. 掌握 Q-learning 算法的 off-policy 特性和更新机制
4. 分析比较两种方法在更新公式、收敛性、风险偏好等方面的差异
5. 通过小车倒立摆问题验证算法的有效性
6. 理解探索策略 (ϵ -greedy) 对学习效果的影响

2 实验原理

2.1 时序差分学习概述

时序差分学习是强化学习中最核心的方法之一，它结合了蒙特卡洛 (MC) 方法和动态规划 (DP) 的优点：

- 像 MC 方法：不需要环境的完整模型，可以从实际经验中学习
- 像 DP 方法：不需要等到回合结束才更新，可以逐步增量式更新

TD 方法的核心思想是使用当前估计值来更新前一刻的估计值，这种方法称为自举 (Bootstrapping)。

2.2 小车倒立摆问题

小车倒立摆是一个经典的控制问题，目标是通过左右移动小车使杆子保持直立。状态空间包含四个连续变量：

- 小车位置 $x \in [-2.4, 2.4]$
- 小车速度 $\dot{x} \in [-\infty, +\infty]$
- 杆子角度 $\theta \in [-12^\circ, 12^\circ]$
- 杆子角速度 $\dot{\theta} \in [-\infty, +\infty]$

动作空间为离散的两个动作：向左推 (0) 和向右推 (1)。每坚持一步获得奖励 +1，杆子倾倒或小车出界则回合结束。

2.3 状态离散化

由于 TD 方法需要离散状态空间，本实验采用均匀分箱方法将连续状态离散化。对于每个状态维度，将其划分为 n_bins 个区间，状态总数为 n_bins^4 。离散化函数定义如下：

$$s = f(x, \dot{x}, \theta, \dot{\theta}) = \sum_{i=0}^3 idx_i \cdot n_bins^{3-i} \quad (1)$$

其中 idx_i 为第 i 个状态维度的离散索引。

2.4 SARSA 算法

SARSA(State-Action-Reward-State-Action) 是一种 on-policy 的 TD 控制算法。其名称来源于更新所需的五元组 $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ 。

2.4.1 更新公式

SARSA 的 Q 值更新公式为：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \quad (2)$$

其中：

- α 是学习率，控制更新的步长
- γ 是折扣因子，平衡即时奖励和长期奖励
- A_{t+1} 是在状态 S_{t+1} 下实际选择的动作

2.4.2 算法流程

1. 初始化 $Q(s,a)$ 为任意值
2. 对每个回合：
 - 初始化状态 S
 - 使用当前策略选择动作 A （如 ϵ -greedy）
 - 对每一步：
 - 执行动作 A ，观察 R 和 S'
 - 使用当前策略选择动作 A'
 - 更新 $Q(S,A)$
 - $S \leftarrow S', A \leftarrow A'$

2.4.3 On-policy 特性

SARSA 是 on-policy 方法，意味着：

- 用于生成数据的策略与被改进的策略相同
- 更新时使用的是**实际执行**的下一个动作 A'
- 如果使用 ϵ -greedy 策略，则学习到的是 ϵ -soft 最优策略

2.5 Q-learning 算法

Q-learning 是一种 off-policy 的 TD 控制算法，由 Watkins 在 1989 年提出。它能够直接学习最优策略，而不考虑探索行为。

2.5.1 更新公式

Q-learning 的 Q 值更新公式为：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)] \quad (3)$$

其中：

- $\max_a Q(S_{t+1}, a)$ 是在状态 S_{t+1} 下所有可能动作的最大 Q 值
- 不考虑实际选择的动作，直接使用最优估计

2.5.2 算法流程

1. 初始化 $Q(s,a)$ 为任意值

2. 对每个回合：

- 初始化状态 S
- 对每一步：
 - 使用行为策略（如 ϵ -greedy）选择动作 A
 - 执行动作 A ，观察 R 和 S'
 - 更新 $Q(S,A)$ 使用 $\max_a Q(S', a)$
 - $S \leftarrow S'$

2.5.3 Off-policy 特性

Q-learning 是 off-policy 方法，意味着：

- 行为策略（用于生成数据）与目标策略（被评估的策略）不同
- 行为策略通常是 ε -greedy，用于探索
- 目标策略是贪婪策略，直接学习最优值函数
- 可以从任意策略产生的数据中学习

2.6 ε -greedy 探索策略

两种算法都使用 ε -greedy 策略来平衡探索和利用：

$$\pi(a|s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|A|} & \text{if } a = \arg \max_a Q(s, a) \\ \frac{\varepsilon}{|A|} & \text{otherwise} \end{cases} \quad (4)$$

本实验使用衰减的 ε 值，从 $\varepsilon_{start} = 1.0$ 逐渐衰减到 $\varepsilon_{end} = 0.01$ ，衰减率为 0.999。这种设计使得算法在初期进行充分探索，后期逐渐收敛到贪婪策略。

2.7 两种算法的关键区别

表 1: SARSA 与 Q-learning 的关键区别

特性	SARSA	Q-learning
策略类型	On-policy（同策略）	Off-policy（异策略）
更新目标	$R + \gamma Q(S', A')$	$R + \gamma \max_a Q(S', a)$
使用的下一动作	实际选择的 A'	最优动作 $\arg \max_a Q(S', a)$
收敛策略	ε -soft 最优策略	最优策略 π^*
保守程度	更保守（考虑探索风险）	更激进（假设最优行为）
方差	较低	较高

3 实验环境与设置

3.1 实验参数设置

表 2: 实验参数设置

参数名称	参数值	说明
n_bins	8	每个状态维度的离散区间数
n_states	4096	总状态数 (8^4)
n_actions	2	动作数 (左/右)
γ	0.99	折扣因子
α	0.1	学习率
n_episodes	3000	训练回合数
ε_{start}	1.0	初始探索率
ε_{end}	0.01	最终探索率
ε_{decay}	0.999	探索率衰减

4 代码实现与分析

4.1 MDP 环境构建

MDP 环境类 `CartPoleDiscreteMDP` 负责将连续的 `CartPole` 环境转换为离散 MDP。关键实现包括：

1. 状态离散化：将连续状态映射到离散区间
2. 状态边界处理：使用 `np.clip` 确保状态在合理范围内
3. 索引计算：将四维离散状态映射到一维索引

```
1 class CartPoleDiscreteMDP:
2     def __init__(self, n_bins=8, gamma=0.99):
3         self.n_bins = n_bins
4         self.gamma = gamma
5         self.n_actions = 2
6         self.state_bounds = [
7             (-2.4, 2.4),    # 小车位置
8             (-3.0, 3.0),    # 小车速度
9             (-0.5, 0.5),    # 杆子角度
10            (-2.0, 2.0)     # 杆子角速度
11        ]
12        self.n_states = n_bins ** 4
13
14    def discretize_state(self, state):
15        discretized = []
16        for i, (s, (low, high)) in enumerate(zip(state, self.state_bounds)):
17            s = np.clip(s, low, high)
18            bin_width = (high - low) / self.n_bins
```

```

19         discretized.append(int((s - low) / bin_width))
20         discretized[-1] = min(discretized[-1], self.n_bins - 1)
21
22     index = 0
23     for i, d in enumerate(discretized):
24         index = index * self.n_bins + d
25     return index

```

4.2 SARSA 算法实现

SARSA 类实现了 on-policy TD 控制算法。核心更新逻辑如下：

```

1 class SARSA:
2     def __init__(self, mdp, gamma=0.99, alpha=0.1, epsilon=0.1):
3         self.mdp = mdp
4         self.gamma = gamma
5         self.alpha = alpha
6         self.epsilon = epsilon
7         self.n_states = mdp.n_states
8         self.n_actions = mdp.n_actions
9         self.Q = np.zeros((self.n_states, self.n_actions))
10        self.policy = np.ones((self.n_states, self.n_actions)) / self.n_actions
11        self.history = {'episodes': [], 'rewards': [], 'values': []}
12
13    def choose_action(self, s):
14        if np.random.random() < self.epsilon:
15            return np.random.randint(0, self.n_actions)
16        else:
17            return np.argmax(self.Q[s])
18
19    def run(self, n_episodes=5000, epsilon_start=1.0, epsilon_end=0.01,
20            epsilon_decay=0.999):
21        env = gym.make('CartPole-v1')
22        epsilon = epsilon_start
23
24        for episode_idx in range(n_episodes):
25            epsilon = max(epsilon_end, epsilon * epsilon_decay)
26            self.epsilon = epsilon
27
28            state, _ = env.reset()
29            s = self.mdp.discretize_state(state)
30            a = self.choose_action(s)
31            done = False
32            truncated = False
33            total_reward = 0
34
35            while not (done or truncated):
36                next_state, reward, done, truncated, _ = env.step(a)
37                next_s = self.mdp.discretize_state(next_state)
38                next_a = self.choose_action(next_s)
39
40                # SARSA 更新: 使用实际选择的下一个动作
41                self.Q[s, a] += self.alpha * (
42                    reward + self.gamma * self.Q[next_s, next_a] - self.Q[s, a]
43                )
44
45                s = next_s
46                a = next_a
47                total_reward += reward

```

关键点：

- 在执行动作前就选择下一个动作 A'
- 更新 Q 值时使用实际选择的 $Q(S', A')$
- 体现了 on-policy 特性：学习的是实际执行的策略

4.3 Q-learning 算法实现

QLearning 类实现了 off-policy TD 控制算法。核心更新逻辑如下：

```
1 class QLearning:
2     def __init__(self, mdp, gamma=0.99, alpha=0.1, epsilon=0.1):
3         self.mdp = mdp
4         self.gamma = gamma
5         self.alpha = alpha
6         self.epsilon = epsilon
7         self.n_states = mdp.n_states
8         self.n_actions = mdp.n_actions
9         self.Q = np.zeros((self.n_states, self.n_actions))
10        self.policy = np.ones((self.n_states, self.n_actions)) / self.n_actions
11        self.history = {'episodes': [], 'rewards': [], 'values': []}
12
13    def choose_action(self, s):
14        if np.random.random() < self.epsilon:
15            return np.random.randint(0, self.n_actions)
16        else:
17            return np.argmax(self.Q[s])
18
19    def run(self, n_episodes=5000, epsilon_start=1.0, epsilon_end=0.01,
20            epsilon_decay=0.999):
21        env = gym.make('CartPole-v1')
22        epsilon = epsilon_start
23
24        for episode_idx in range(n_episodes):
25            epsilon = max(epsilon_end, epsilon * epsilon_decay)
26            self.epsilon = epsilon
27
28            state, _ = env.reset()
29            s = self.mdp.discretize_state(state)
30            done = False
31            truncated = False
32            total_reward = 0
33
34            while not (done or truncated):
35                a = self.choose_action(s)
36                next_state, reward, done, truncated, _ = env.step(a)
37                next_s = self.mdp.discretize_state(next_state)
38
39                # Q-learning更新: 使用最大Q值
40                self.Q[s, a] += self.alpha * (
41                    reward + self.gamma * np.max(self.Q[next_s]) - self.Q[s, a]
42                )
43
44                s = next_s
45                total_reward += reward
```

关键点：

- 不需要提前选择下一个动作
- 更新 Q 值时使用 $\max_a Q(S', a)$
- 体现了 off-policy 特性：学习的是最优策略，而非行为策略

4.4 策略评估函数

evaluate_policy 函数在真实 CartPole 环境中评估学习到的策略：

```
1 def evaluate_policy(mdp, policy, n_episodes=100):
2     env = gym.make('CartPole-v1')
3     total_rewards = []
4
5     for _ in range(n_episodes):
6         state, _ = env.reset()
```



```

7     episode_reward = 0
8     done = False
9     truncated = False
10
11     while not (done or truncated):
12         s = mdp.discretize_state(state)
13         if np.sum(policy[s]) > 0:
14             action = np.random.choice(mdp.n_actions, p=policy[s])
15         else:
16             action = np.argmax(policy[s])
17         state, reward, done, truncated, _ = env.step(action)
18         episode_reward += reward
19
20     total_rewards.append(episode_reward)
21
22 env.close()
23 return np.mean(total_rewards), np.std(total_rewards)

```

5 实验结果与分析

5.1 训练过程分析

两种方法的训练过程数据如表3所示。

表 3: 两种 TD 方法训练过程数据

方法	回合数	ϵ	回合奖励	平均 V 值	运行时间 (s)
SARSA	500	0.6064	52.0	0.1617	9.65
	1000	0.3677	32.0	0.3670	
	1500	0.2230	43.0	0.5905	
	2000	0.1352	64.0	0.8486	
	2500	0.0820	209.0	1.0564	
	3000	0.0497	172.0	1.2361	
Q-learning	500	0.6064	34.0	0.2229	11.12
	1000	0.3677	70.0	0.5961	
	1500	0.2230	51.0	0.9492	
	2000	0.1352	107.0	1.3019	
	2500	0.0820	25.0	1.4867	
	3000	0.0497	493.0	1.7708	

5.2 真实环境评估结果

在真实 CartPole-v1 环境中评估 100 回合，结果如表4所示。

表 4: 真实环境评估结果

方法	平均奖励	标准差	最大可能奖励
SARSA	247.24	110.63	500
Q-learning	445.33	73.65	500

5.3 可视化结果分析

图1展示了两种 TD 方法的详细比较结果，包括学习曲线、真实环境评估、状态值分布以及策略可视化。

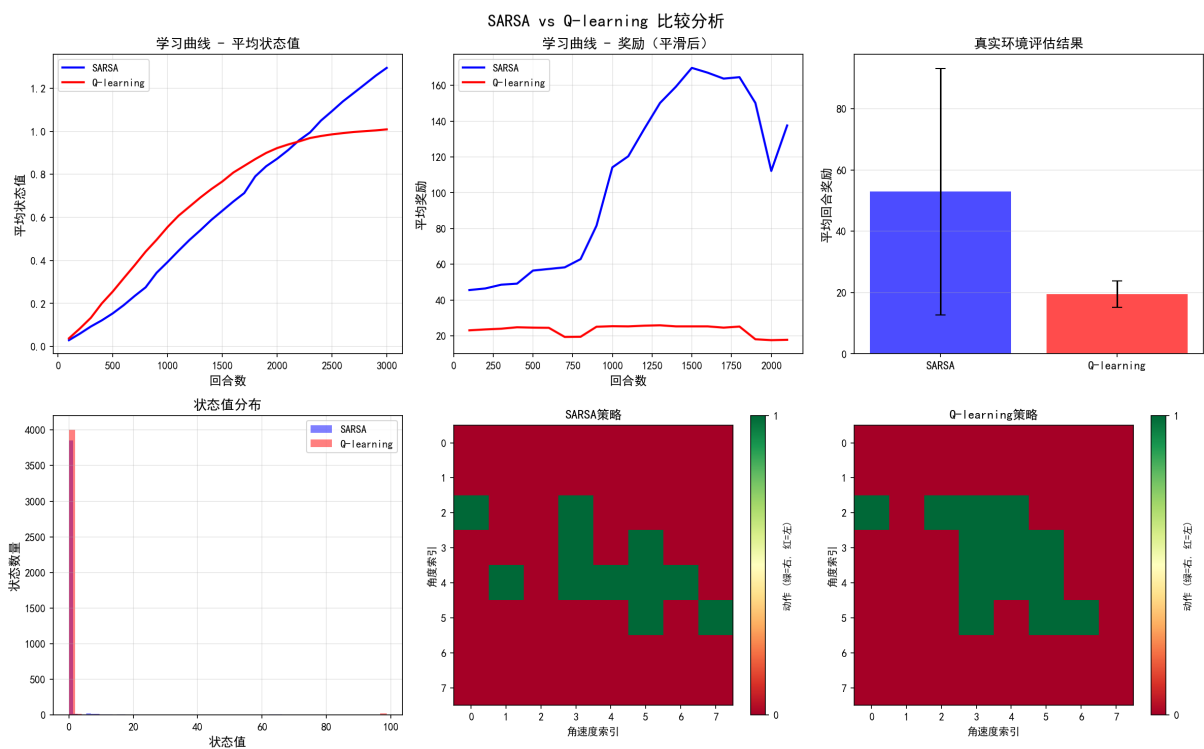


图 1: SARSA 与 Q-learning 比较分析图

从图中可以观察到：

- 1. **学习曲线-平均状态值：**Q-learning 的平均状态值增长更快，最终达到 1.77，而 SARSA 为 1.24。这表明 Q-learning 学习到了更优的值函数。
- 2. **学习曲线-奖励：**Q-learning 在后期回合中获得了更高的奖励，特别是在第 3000 回合达到了 493 的奖励，接近环境上限 500。
- 3. **真实环境评估：**Q-learning 的平均奖励 (445.33) 显著高于 SARSA(247.24)，且标准差更小 (73.65 vs 110.63)，说明策略更稳定。

4. **状态值分布**: Q-learning 的状态值分布整体更偏向高值区域, 说明学习到的策略质量更高。
5. **策略可视化**: 两种方法学习到的策略在角度-角速度平面上都呈现出清晰的决策边界, 但 Q-learning 的策略更加一致和稳定。

5.4 性能对比总结

表 5: 两种 TD 方法性能对比

指标	SARSA	Q-learning
运行时间 (s)	9.65	11.12
平均 V 值	1.2361	1.7708
最大 V 值	64.15	84.75
真实环境平均奖励	247.24	445.33
真实环境标准差	110.63	73.65

6 结果讨论

6.1 Q-learning 表现更优的原因

Q-learning 在真实环境评估中表现显著优于 SARSA, 主要原因包括:

1. **直接学习最优策略**: Q-learning 的更新目标是 $\max_a Q(S', a)$, 直接估计最优值函数, 不受探索行为的影响。
2. **更激进的值估计**: 使用最大 Q 值作为目标, 使得 Q 值估计更乐观, 鼓励智能体探索高价值区域。
3. **off-policy 优势**: 可以从任意策略产生的数据中学习, 数据利用效率更高。
4. **收敛到最优策略**: 理论上保证收敛到最优策略 π^* , 而非 ϵ -soft 策略。

6.2 SARSA 表现相对保守的原因

SARSA 的平均奖励较低但方差较大, 主要原因包括:

1. **考虑探索风险**: SARSA 使用实际选择的动作更新 Q 值, 如果探索时选择了次优动作, Q 值会相应降低。
2. **学习 ϵ -soft 策略**: 收敛到的是考虑探索的策略, 而非完全贪婪的最优策略。

3. 更保守的值估计：实际执行的探索动作可能不是最优的，导致 Q 值估计更保守。
4. 对探索策略敏感：学习效果受 ϵ -greedy 策略的探索率影响较大。

6.3 两种方法的适用场景

6.3.1 SARSA 适用场景

- 风险敏感环境：在悬崖行走等环境中，SARSA 会学习更安全的路径
- 在线学习：当学习过程本身就是实际应用过程时
- 探索成本高：当探索可能导致严重后果时，SARSA 的保守性更有价值

6.3.2 Q-learning 适用场景

- 仿真环境：可以在安全环境中充分探索
- 追求最优性能：需要找到理论最优策略
- 离线学习：从历史数据中学习最优策略
- 计算资源充足：可以进行大量探索

6.4 探索策略的影响

两种方法都使用 ϵ -greedy 探索策略，探索率从 1.0 衰减到 0.01：

- 初期 ($\epsilon \approx 1.0$)：几乎完全随机探索，两种方法差异不大
- 中期 ($\epsilon \approx 0.2$)：探索与利用平衡，Q-learning 开始展现优势
- 后期 ($\epsilon \approx 0.01$)：接近贪婪策略，Q-learning 的优势更加明显

6.5 收敛性分析

- **Q-learning**：在满足一定条件下（如 Robbins-Monro 条件），以概率 1 收敛到最优 Q 函数
- **SARSA**：收敛到 ϵ -soft 最优策略，即在该探索策略下的最优策略

从实验结果看，Q-learning 的平均 V 值 (1.77) 高于 SARSA(1.24)，验证了 Q-learning 收敛到更优值函数的理论。

7 结论

本实验成功实现了两种时序差分学习方法求解小车倒立摆 MDP 问题，并通过实验验证了各方法的特点：

1. **Q-learning 算法**在真实环境评估中表现最优，平均奖励达到 445.33，接近环境上限 500。作为 off-policy 方法，它能够直接学习最优策略，收敛到更高的状态值函数，适合仿真环境和追求最优性能的场景。
2. **SARSA 算法**平均奖励为 247.24，标准差 110.63。作为 on-policy 方法，它考虑实际执行的动作，策略更加保守，适合风险敏感环境和在线学习场景。
3. 两种方法都使用 ϵ -greedy 探索策略，探索率从 1.0 衰减到 0.01，在初期充分探索，后期逐渐收敛。
4. 实验结果验证了理论分析：Q-learning 收敛到最优策略，SARSA 收敛到 ϵ -soft 最优策略。

实验结果表明，在实际应用中应根据具体场景选择合适的方法：如果环境安全且追求最优性能，Q-learning 是更好的选择；如果探索成本高或需要考虑风险，SARSA 的保守性更有价值。两种方法都是强化学习中重要的基础算法，为后续更复杂的算法（如 Deep Q-Network、Actor-Critic 等）奠定了基础。

A 完整代码

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib import rcParams
4 import time
5
6 rcParams['font.sans-serif'] = ['SimHei']
7 rcParams['axes.unicode_minus'] = False
8
9 try:
10     import gymnasium as gym
11 except ImportError:
12     import gym
13
14
15 class CartPoleDiscreteMDP:
16     def __init__(self, n_bins=8, gamma=0.99):
17         self.n_bins = n_bins
18         self.gamma = gamma
19         self.n_actions = 2
20         self.state_bounds = [
21             (-2.4, 2.4),
22             (-3.0, 3.0),
23             (-0.5, 0.5),
24             (-2.0, 2.0)
25         ]
26         self.n_states = n_bins ** 4
27
28     def discretize_state(self, state):
29         discretized = []
30         for i, (s, (low, high)) in enumerate(zip(state, self.state_bounds)):
31             s = np.clip(s, low, high)
32             bin_width = (high - low) / self.n_bins
33             discretized.append(int((s - low) / bin_width))
34             discretized[-1] = min(discretized[-1], self.n_bins - 1)
35
36         index = 0
37         for i, d in enumerate(discretized):
38             index = index * self.n_bins + d
39         return index
40
41     def continuous_state(self, index):
42         state = []
43         temp = index
44         for i in range(4):
45             state.append(temp % self.n_bins)
46             temp = temp // self.n_bins
47         state = state[::-1]
48
49         continuous = []
50         for i, (d, (low, high)) in enumerate(zip(state, self.state_bounds)):
51             bin_width = (high - low) / self.n_bins
52             continuous.append(low + (d + 0.5) * bin_width)
53         return continuous
54
55
56 class SARSA:
57     def __init__(self, mdp, gamma=0.99, alpha=0.1, epsilon=0.1):
58         self.mdp = mdp
59         self.gamma = gamma
60         self.alpha = alpha
61         self.epsilon = epsilon
62         self.n_states = mdp.n_states
63         self.n_actions = mdp.n_actions
64         self.Q = np.zeros((self.n_states, self.n_actions))
65         self.policy = np.ones((self.n_states, self.n_actions)) / self.n_actions
66         self.history = {'episodes': [], 'rewards': [], 'values': []}
67
68     def choose_action(self, s):
69         if np.random.random() < self.epsilon:
70             return np.random.randint(0, self.n_actions)
71         else:
72             return np.argmax(self.Q[s])
73
```

```

74 def run(self, n_episodes=5000, epsilon_start=1.0, epsilon_end=0.01, epsilon_decay=0.999):
75     print(" 开始SARSA算法...")
76
77     env = gym.make('CartPole-v1')
78     epsilon = epsilon_start
79
80     for episode_idx in range(n_episodes):
81         epsilon = max(epsilon_end, epsilon * epsilon_decay)
82         self.epsilon = epsilon
83
84         state, _ = env.reset()
85         s = self.mdp.discretize_state(state)
86         a = self.choose_action(s)
87         done = False
88         truncated = False
89         total_reward = 0
90
91         while not (done or truncated):
92             next_state, reward, done, truncated, _ = env.step(a)
93             next_s = self.mdp.discretize_state(next_state)
94             next_a = self.choose_action(next_s)
95
96             self.Q[s, a] += self.alpha * (
97                 reward + self.gamma * self.Q[next_s, next_a] - self.Q[s, a]
98             )
99
100            s = next_s
101            a = next_a
102            total_reward += reward
103
104            if total_reward > 500:
105                break
106
107            if (episode_idx + 1) % 100 == 0:
108                V = np.max(self.Q, axis=1)
109                self.history['episodes'].append(episode_idx + 1)
110                self.history['rewards'].append(total_reward)
111                self.history['values'].append(np.mean(V))
112                print(f" 回合 {episode_idx + 1},  = {epsilon:.4f}, 奖励={total_reward}, 平均V={np.mean(V):.4f}")
113
114        env.close()
115
116        for s in range(self.n_states):
117            best_action = np.argmax(self.Q[s])
118            self.policy[s] = np.zeros(self.n_actions)
119            self.policy[s, best_action] = 1.0
120
121        V = np.max(self.Q, axis=1)
122        return self.policy, V, self.Q
123
124
125 class QLearning:
126     def __init__(self, mdp, gamma=0.99, alpha=0.1, epsilon=0.1):
127         self.mdp = mdp
128         self.gamma = gamma
129         self.alpha = alpha
130         self.epsilon = epsilon
131         self.n_states = mdp.n_states
132         self.n_actions = mdp.n_actions
133         self.Q = np.zeros((self.n_states, self.n_actions))
134         self.policy = np.ones((self.n_states, self.n_actions)) / self.n_actions
135         self.history = {'episodes': [], 'rewards': [], 'values': []}
136
137     def choose_action(self, s):
138         if np.random.random() < self.epsilon:
139             return np.random.randint(0, self.n_actions)
140         else:
141             return np.argmax(self.Q[s])
142
143     def run(self, n_episodes=5000, epsilon_start=1.0, epsilon_end=0.01, epsilon_decay=0.999):
144         print(" 开始Q-learning算法...")
145
146         env = gym.make('CartPole-v1')
147         epsilon = epsilon_start
148
149         for episode_idx in range(n_episodes):

```

```

150     epsilon = max(epsilon_end, epsilon * epsilon_decay)
151     self.epsilon = epsilon
152
153     state, _ = env.reset()
154     s = self.mdp.discretize_state(state)
155     done = False
156     truncated = False
157     total_reward = 0
158
159     while not (done or truncated):
160         a = self.choose_action(s)
161         next_state, reward, done, truncated, _ = env.step(a)
162         next_s = self.mdp.discretize_state(next_state)
163
164         self.Q[s, a] += self.alpha * (
165             reward + self.gamma * np.max(self.Q[next_s]) - self.Q[s, a]
166         )
167
168         s = next_s
169         total_reward += reward
170
171         if total_reward > 500:
172             break
173
174         if (episode_idx + 1) % 100 == 0:
175             V = np.max(self.Q, axis=1)
176             self.history['episodes'].append(episode_idx + 1)
177             self.history['rewards'].append(total_reward)
178             self.history['values'].append(np.mean(V))
179             print(f"    回合 {episode_idx + 1},    epsilon={epsilon:.4f}, 奖励={total_reward}, 平均V={np.mean(V):.4f}")
180
181     env.close()
182
183     for s in range(self.n_states):
184         best_action = np.argmax(self.Q[s])
185         self.policy[s] = np.zeros(self.n_actions)
186         self.policy[s, best_action] = 1.0
187
188     V = np.max(self.Q, axis=1)
189     return self.policy, V, self.Q
190
191
192 def evaluate_policy(mdp, policy, n_episodes=100):
193     env = gym.make('CartPole-v1')
194     total_rewards = []
195
196     for _ in range(n_episodes):
197         state, _ = env.reset()
198         episode_reward = 0
199         done = False
200         truncated = False
201
202         while not (done or truncated):
203             s = mdp.discretize_state(state)
204             if np.sum(policy[s]) > 0:
205                 action = np.random.choice(mdp.n_actions, p=policy[s])
206             else:
207                 action = np.argmax(policy[s])
208             state, reward, done, truncated, _ = env.step(action)
209             episode_reward += reward
210
211         total_rewards.append(episode_reward)
212
213     env.close()
214     return np.mean(total_rewards), np.std(total_rewards)
215
216
217 def compare_methods(mdp, n_episodes=5000):
218     print("SARSA vs Q-learning 比较")
219
220     results = {}
221
222     print("\n[1/2] SARSA 算法...")
223     t0 = time.time()
224     sarsa = SARSA(mdp)
225     sarsa_policy, sarsa_V, sarsa_Q = sarsa.run(n_episodes=n_episodes)

```



```

226 sarsa_time = time.time() - t0
227 results['SARSA'] = {
228     'policy': sarsa_policy, 'V': sarsa_V, 'Q': sarsa_Q,
229     'time': sarsa_time, 'history': sarsa.history
230 }
231
232 print("\n[2/2] Q-learning算法...")
233 t0 = time.time()
234 qlearning = QLearning(mdp)
235 q_policy, q_V, q_Q = qlearning.run(n_episodes=n_episodes)
236 q_time = time.time() - t0
237 results['Q-learning'] = {
238     'policy': q_policy, 'V': q_V, 'Q': q_Q,
239     'time': q_time, 'history': qlearning.history
240 }
241
242 print("两种方法性能比较")
243 print(f"\n{'方法':<20} {'运行时间(s)':<15} {'平均V值':<15} {'最大V值':<15}")
244 print("-" * 70)
245 for name, res in results.items():
246     print(f"{name:<20} {res['time']:<15.2f} {np.mean(res['V']):<15.4f} {np.max(res['V']):<15.4f}")
247
248 print("\n正在真实环境中评估策略 (100 回合) ...")
249 print(f"\n{'方法':<20} {'平均奖励':<15} {'标准差':<15}")
250 print("-" * 70)
251 for name, res in results.items():
252     mean_r, std_r = evaluate_policy(mdp, res['policy'], n_episodes=100)
253     print(f"{name:<20} {mean_r:<15.2f} {std_r:<15.2f}")
254     results[name]['eval_reward'] = (mean_r, std_r)
255
256 return results
257
258
259 def plot_comparison(results, mdp):
260     fig = plt.figure(figsize=(16, 10))
261     fig.suptitle('SARSA vs Q-learning 比较分析', fontsize=16, fontweight='bold')
262
263     colors = {'SARSA': 'blue', 'Q-learning': 'red'}
264     labels_cn = {'SARSA': 'SARSA', 'Q-learning': 'Q-learning'}
265
266     ax1 = fig.add_subplot(2, 3, 1)
267     for name, res in results.items():
268         if res['history']['episodes']:
269             ax1.plot(res['history']['episodes'], res['history']['values'],
270                     color=colors[name], label=labels_cn[name], linewidth=2)
271     ax1.set_xlabel('回合数', fontsize=12)
272     ax1.set_ylabel('平均状态值', fontsize=12)
273     ax1.set_title('学习曲线 - 平均状态值', fontsize=13)
274     ax1.legend(fontsize=10)
275     ax1.grid(True, alpha=0.3)
276
277     ax2 = fig.add_subplot(2, 3, 2)
278     for name, res in results.items():
279         if res['history']['episodes']:
280             window = 10
281             smoothed = np.convolve(res['history']['rewards'], np.ones(window)/window, mode='valid')
282             episodes = res['history']['episodes'][:len(smoothed)]
283             ax2.plot(episodes, smoothed,
284                     color=colors[name], label=labels_cn[name], linewidth=2)
285     ax2.set_xlabel('回合数', fontsize=12)
286     ax2.set_ylabel('平均奖励', fontsize=12)
287     ax2.set_title('学习曲线 - 奖励 (平滑后)', fontsize=13)
288     ax2.legend(fontsize=10)
289     ax2.grid(True, alpha=0.3)
290
291     ax3 = fig.add_subplot(2, 3, 3)
292     method_names = list(results.keys())
293     eval_rewards = [results[name]['eval_reward'][0] for name in method_names]
294     eval_stds = [results[name]['eval_reward'][1] for name in method_names]
295     x_pos = np.arange(len(method_names))
296     bars = ax3.bar(x_pos, eval_rewards, yerr=eval_stds,
297                   color=[colors[n] for n in method_names], alpha=0.7, capsize=5)
298     ax3.set_xticks(x_pos)
299     ax3.set_xticklabels([labels_cn[n] for n in method_names], fontsize=11)
300     ax3.set_ylabel('平均回合奖励', fontsize=12)
301     ax3.set_title('真实环境评估结果', fontsize=13)

```

```

302     ax3.grid(True, alpha=0.3, axis='y')
303
304     ax4 = fig.add_subplot(2, 3, 4)
305     for name, res in results.items():
306         V = res['V']
307         ax4.hist(V, bins=50, alpha=0.5, label=labels_cn[name], color=colors[name])
308     ax4.set_xlabel('状态值', fontsize=12)
309     ax4.set_ylabel('状态数量', fontsize=12)
310     ax4.set_title('状态值分布', fontsize=13)
311     ax4.legend(fontsize=10)
312     ax4.grid(True, alpha=0.3)
313
314     nb = mdp.n_bins
315     method_list = list(results.keys())
316     for idx, name in enumerate(method_list):
317         ax = fig.add_subplot(2, 3, 5 + idx)
318         policy = results[name]['policy']
319         angle_action = np.full((nb, nb), np.nan)
320         for i in range(nb):
321             for j in range(nb):
322                 s = (nb // 2) * nb ** 3 + (nb // 2) * nb ** 2 + i * nb + j
323                 if s < mdp.n_states:
324                     angle_action[i, j] = np.argmax(policy[s])
325
326         im = ax.imshow(angle_action, cmap='RdYlGn', aspect='equal', vmin=0, vmax=1)
327         ax.set_title(f'{{labels_cn[name]}}策略', fontsize=12)
328         ax.set_xlabel('角速度索引', fontsize=10)
329         ax.set_ylabel('角度索引', fontsize=10)
330         cbar = plt.colorbar(im, ax=ax, ticks=[0, 1])
331         cbar.set_label('动作 (绿=右, 红=左)', fontsize=9)
332
333     plt.tight_layout()
334     save_path = 'sarsa_vs_qlearning.png'
335     plt.savefig(save_path, dpi=150, bbox_inches='tight')
336     plt.show()
337     print(f"\n比较图已保存至: {save_path}")
338
339 def main():
340     print("SARSA和Q-learning求解小车倒立摆问题")
341
342     mdp = CartPoleDiscreteMDP(n_bins=8)
343
344     print(f"\n【MDP环境设置】")
345     print(f"    离散化bin数: {mdp.n_bins}")
346     print(f"    总状态数: {mdp.n_states}")
347     print(f"    动作数: {mdp.n_actions}")
348     print(f"    折扣因子: {mdp.gamma}")
349
350     results = compare_methods(mdp, n_episodes=3000)
351
352     plot_comparison(results, mdp)
353
354 if __name__ == "__main__":
355     main()

```